Intelligent Model Design

6-7 Hour End-Term Survival Guide

Built from every supplied lecture PDF, lab sheet, and the December 2025 question paper

This is a revision guide, not a replacement for sleep, water, and calm handwriting. The sample pattern is inferred from the supplied 40-mark paper; your 100-mark end-term paper may use a different section split. The concepts and numerical methods come directly from the supplied course material.

> **Your job is not to become a mathematician tonight. Your job is to recognize the question type, write the correct formula, substitute carefully, and collect marks.**

Start here: the 6-7 hour plan

Time	What to do	Concrete output
0:00-0:20	Read this plan, the notation section, and the old-paper pattern	Know what is likely and how to show work
0:20-1:35	CNN, convolution, output sizes, parameters, receptive field, pooling	Solve Workbook Q1-Q12 without looking
1:35-2:25	RNN, LSTM, GRU and sequence parameters	Solve Workbook Q13-Q19
2:25-3:05	Attention, Transformer and ViT	Solve Workbook Q20-Q25
3:05-3:50	Autoencoder, GAN, Pix2Pix, CycleGAN and adversarial attacks	Solve Workbook Q26-Q33
3:50-4:20	Regularization, metrics, XAI, transfer learning and data bias	Revise high-yield comparison tables
4:20-5:50	Attempt Mock Paper 1 under time pressure	Mark it immediately and list mistakes
5:50-6:30	Redo every wrong numerical; scan Mock Papers 2 and 3	One clean solution per weak question type
Last 20-30 min	Read only the formula sheet and architecture cues	Stop learning new material

Three rules for tonight

- **Active recall:** cover the answer and say or write it. Reading feels fluent but does not prove recall.
- **Numerical format:** write Given, Formula, Substitution, Answer. Even a calculation error can still earn method marks.
- **Two-minute rule:** if stuck for two minutes, identify the question type and write the relevant formula. Then move on and return later.

What the previous paper tells us

The supplied paper was a 40-mark, two-hour exam. It sampled nearly the whole course and mixed short definitions, architecture explanations, and small calculations. Its strongest signals are below.

Repeated or high-value area	What was asked	What you must be able to do
Training basics	Epoch, batch size, iterations	Compute batches and total updates
CNN	Dilated convolution, Inception, ResNet	Output size, receptive field, parameters, architecture purpose
ViT and Transformer	Patch count and encoder flow	Patch arithmetic and attention sequence
RNN family	Vanishing gradients and GRU gates	Explain gates and compute a cell update

Repeated or high-value area	What was asked	What you must be able to do
Segmentation	Encoder-decoder and U-Net	Explain skip connections and spatial recovery
Generative models	GAN loop, Pix2Pix, CycleGAN, autoencoder	Compare paired/unpaired training and calculate patches/parameters
Transfer, XAI, LLM	Applications and suitability	Give definition, mechanism, example, limitation

> **Most likely numerical families: iterations, convolution output, dilation, trainable parameters, ViT patches, softmax/cross-entropy, attention, recurrent cell updates, GAN losses, and classification metrics.**

Part I - Math without panic

The mark-scoring numerical routine

For every problem, use **GFSA**:

1. **Given:** copy values with symbols and units.
2. **Formula:** write the general formula before numbers.
3. **Substitution:** insert numbers with brackets.
4. **Answer:** box the result and include shape, count, or unit.

Example: 100,000 samples, batch size 100, 10 epochs.

```
Given: M = 100000 samples, B = 100 samples/batch, E = 10 epochs
Batches per epoch = ceil(M / B) = ceil(100000 / 100) = 1000
Total updates = E * batches per epoch = 10 * 1000 = 10000
Answer: 1000 batches per epoch and 10000 parameter updates.
```

Essential operations and every symbol

Symbol or operation	Meaning	Plain-language explanation				
`x`	Input	The data fed to a model				
`y`	True target	Correct label or desired output				
`y_hat`	Predicted output	Model's estimate; the hat means estimated				
`w`, `W`	Weight	Learned multiplier; uppercase usually means a matrix				
`b`	Bias	Learned offset added after weighted input				
`theta`	All parameters	Shorthand for every trainable weight and bias				
`L`	Loss	Number measuring how wrong the model is				

Symbol or operation	Meaning	Plain-language explanation				
`N` or `M`	Count	Number of samples, tokens, positions, or features; define it in your answer				
`floor(a)`	Round down	`floor(3.9) = 3`				
`ceil(a)`	Round up	`ceil(3.1) = 4`; useful for an incomplete last batch				
`exp(a)`	Exponential	`e` raised to power `a`; used by sigmoid and softmax				
`ln(a)` or `log(a)`	Natural logarithm	Inverse of `exp`; used in cross-entropy and GAN loss				
`sqrt(a)`	Square root	Positive number whose square is `a`				
`sum`	Add many values	Sigma notation in textbooks means repeated addition				
`T` as superscript	Transpose	Swaps matrix rows and columns				
`.*`	Element-wise product	Multiply matching positions, not matrix multiplication				
`		v		_1`	L1 norm	Sum of absolute values in vector v
`		v		_2^2`	Squared L2 norm	Sum of squared values in vector v

Shape discipline

Always write tensor shapes. A color image is commonly `H x W x C`, where:

- `H` = height in pixels.
- `W` = width in pixels.
- `C` = channels; `C = 1` for grayscale and usually `C = 3` for RGB.
- A batch adds `B`, so a channels-last batch is `B x H x W x C`.

If a question does not state padding, stride, bias, overlap, or token convention, state your assumption before calculating. This is not weakness; it makes the answer defensible.

Part II - Foundations, data and training

Intelligent models

A traditional rule system follows manually written conditions. An intelligent model learns a mapping or policy from data. It can generalize to unseen inputs, but its behavior depends on data quality, objective, architecture, and deployment context.

Sensitive applications in the slides include healthcare, cybersecurity, autonomous systems, finance, and defense. Strong answers mention:

- context-aware input and domain knowledge;
- representative and diverse data;
- uncertainty or anomaly review;
- human oversight where failure is costly;
- monitoring after deployment because data distributions change.

Dataset dimensions

Type	Typical shape	Examples
1D	rows x features, or time x channels	Tabular records, ECG, audio waveform, spectral series
2D	height x width, with optional channels	Grayscale/RGB images, X-rays, maps
3D	depth x height x width, often with channels	CT/MRI volume, video volume, hyperspectral cube

The phrase 3D can be ambiguous: an RGB image has three channels but is usually treated as 2D spatial data; a medical scan has three spatial axes.

Epoch, batch and iteration

- **Sample:** one training example.
- **Batch:** subset processed before one optimizer update.
- **Batch size `B`:** number of samples per batch.
- **Iteration/update:** one forward pass, loss, backward pass, and parameter update for a batch.
- **Epoch `E`:** one pass through the training set.

```
batches_per_epoch = ceil(M / B)
total_updates = E * ceil(M / B)

M = number of training samples
B = batch size
E = number of epochs
ceil = round upward when the last batch is incomplete
```

Bias in intelligent models

Bias	Source	Example	Mitigation
Training/data bias	Unrepresentative collection or labels	Face dataset underrepresents a group	Balanced sampling, data audit, better labels
Algorithmic bias	Objective, features, threshold, or model amplifies disparity	Accuracy-only objective ignores minority errors	Fairness metrics, threshold review, constrained objectives
Cognitive bias	Human assumptions enter design or interpretation	Confirmation bias when selecting evidence	Diverse teams, blind evaluation, documented decisions

Do not claim that deleting a protected attribute automatically removes bias; correlated features can act as proxies. Fairness is a lifecycle: collect, measure, mitigate, document, monitor.

Part III - Convolutional neural networks

What convolution does

A filter or kernel slides over local image regions. At each location, multiply corresponding entries and add them. A learned bias may then be added. Different filters learn different patterns such as edges, textures, and object parts.

Terminology:

- `K_h`, `K_w`: kernel height and width.
- `C_in`: number of input channels.
- `C_out`: number of filters and therefore output channels.
- `S`: stride, the movement per step.
- `P`: zero-padding on each side.
- `D`: dilation, spacing between kernel elements.
- `H_in`, `W_in`: input height and width.
- `H_out`, `W_out`: output height and width.

Convolution output size

```
K_eff = D * (K - 1) + 1
N_out = floor((N_in + 2P - D(K - 1) - 1) / S) + 1

K_eff = effective kernel width or height
D = dilation rate; D = 1 is ordinary convolution
K = physical kernel size
N_in = input width or height
P = padding on each side
S = stride
N_out = output width or height
```

Apply the equation separately to height and width. Output depth is `C_out`.

Quick cases:

- `valid` convolution means `P = 0`.
- For odd `K`, stride 1, dilation 1, same-size output uses `P = (K - 1)/2`.
- A `1 x 1` convolution changes channel depth and mixes channels without changing spatial size when `S = 1, P = 0`.

Worked dilation example from the old-paper style

Input `20 x 20`, kernel `5 x 5`, stride 1, no padding.

Dilation	Effective kernel	Output width/height	Output map
1	`1(5-1)+1 = 5`	`20-5+1 = 16`	`16 x 16`
2	`2(5-1)+1 = 9`	`20-9+1 = 12`	`12 x 12`
3	`3(5-1)+1 = 13`	`20-13+1 = 8`	`8 x 8`

Dilation enlarges receptive field without adding kernel weights, but the output shrinks when padding is not increased.

CNN trainable parameters

```
conv_parameters = (K_h * K_w * C_in + 1) * C_out
dense_parameters = (N_in + 1) * N_out

The +1 represents one bias for each output channel or neuron.
If the problem says no bias, remove the +1 term.
Pooling and standard activation layers have zero trainable parameters.
```

Example: `3 x 3`, 3 input channels, 32 filters:

```
(3 * 3 * 3 + 1) * 32 = 28 * 32 = 896 parameters
```

Important distinction: parameter count does not depend on input height or width because weights are shared. Computation and activation memory do depend on spatial size.

Pooling

- **Max pooling:** keeps the largest value; emphasizes strongest detected feature.
- **Average pooling:** takes the mean; smooths the region.
- Pooling reduces spatial dimensions, computation, and sensitivity to small shifts.
- Global average pooling returns one average per channel and often replaces large fully connected layers.

For pooling of size `K`, stride `S`, padding `P`, use the same output-size formula with dilation 1.

Receptive field

The receptive field is the region of the original input that can affect one output unit. For layers in sequence:

```
jump_l = jump_(l-1) * S_l
RF_l = RF_(l-1) + (K_eff_l - 1) * jump_(l-1)

Start with RF_0 = 1 and jump_0 = 1.
jump means spacing in original-input pixels between neighboring features.
```

Three `3 x 3`, stride-1 convolutions give receptive fields `3`, `5`, then `7`. They use `3 * 9C^2 = 27C^2` weights if all channel widths are `C`, compared with `49C^2` for one `7 x 7` convolution, and add more nonlinearities.

Classic CNN architectures

Model	Memory cue	Exam-worthy idea
LeNet-5	Early digit CNN	Conv-pool stacks followed by fully connected layers
AlexNet	2012 ImageNet breakthrough	ReLU, GPU training, dropout, data augmentation; large early kernels
ZFNet	AlexNet refinement	Visualization-guided filter/stride improvements
VGG	Repeated small filters	Deep uniform stacks of `3 x 3` convolution
Network in Network	Micro-network per location	`1 x 1` convolution and global average pooling
GoogLeNet/Inception	Parallel branches	Multi-scale filters and `1 x 1` bottlenecks
ResNet	Skip addition	Learn residual `F(x)` so `H(x) = F(x) + x`
DenseNet	Concatenate earlier features	Feature reuse and strong gradient paths
ResNeXt	Repeated grouped branches	Cardinality as another capacity dimension
SENet	Channel attention	Squeeze global context, then excite useful channels

AlexNet arithmetic cues

The lecture uses a `227 x 227 x 3` input. Conv1 has 96 filters, `11 x 11`, stride 4, no padding:

```
output = floor((227 - 11) / 4) + 1 = 55
shape = 55 x 55 x 96
parameters = (11 * 11 * 3 + 1) * 96 = 34944
```

A `3 x 3`, stride-2 pool changes 55 to 27. Conv2 keeps 27 with a `5 x 5` kernel by using padding 2. The final convolutional output is commonly `6 x 6 x 256`, flattened to `9216` before fully connected layers.

Inception

An Inception module applies parallel paths such as `1 x 1`, `3 x 3`, `5 x 5`, and pooling, then concatenates their channels. It captures multiple scales. `1 x 1` bottlenecks first reduce channels, making later large kernels cheaper.

If an input has depth 256 and branch outputs have 128, 192, 96, and an unchanged 256-channel pooled branch, concatenated depth is:

```
128 + 192 + 96 + 256 = 672 channels
```

ResNet

```
H(x) = F(x) + x

x = block input and identity shortcut
F(x) = learned residual transformation
H(x) = block output
```

The shortcut gives gradients a direct route and makes very deep networks easier to optimize. Degradation of a plain deep network means training error can become worse as depth increases; it is not simply overfitting. If dimensions differ, a projection shortcut, often `1 x 1`, aligns them. A bottleneck block uses `1 x 1 -> 3 x 3 -> 1 x 1` to reduce, process, and restore channels.

Image segmentation

- **Semantic segmentation:** assign a class to every pixel; two cars share the same class.
- **Instance segmentation:** distinguish individual objects of the same class.
- **Panoptic segmentation:** combines semantic background and separate object instances.

FCN, encoder-decoder, DeconvNet and U-Net

- Sliding-window classification repeats computation and is slow.
- A fully convolutional network replaces dense layers with convolution so it produces a spatial map.
- The encoder compresses spatial information while increasing semantic abstraction.
- The decoder upsamples to restore resolution.
- Transposed convolution is a learned upsampling operation; it is not literally the inverse of convolution.
- DeconvNet can use recorded max-pooling indices for unpooling.
- U-Net copies high-resolution encoder features into the decoder through skip **concatenations**, helping recover boundaries lost during downsampling.

Do not confuse U-Net skips with ResNet skips: U-Net usually concatenates encoder and decoder features; ResNet adds a block input to a residual.

Dilated convolution

Dilated or atrous convolution inserts gaps between kernel elements. It expands context without increasing the number of kernel parameters or using pooling. It is useful in dense prediction, but large dilation can sample the input sparsely and cause gridding artifacts.

Part IV - Activation functions, loss and regularization

Activation functions

Function	Formula	Range	Main point
Step	0 below threshold, 1 above	0 or 1	Not differentiable; historical perceptron use
Sigmoid	`1 / (1 + exp(-x))`	0 to 1	Probability-like output; saturates and can vanish gradients
Tanh	`(exp(x)-exp(-x))/(exp(x)+exp(-x))`	-1 to 1	Zero-centered but still saturates
ReLU	`max(0, x)`	0 to infinity	Simple and common; can produce dead neurons
Leaky ReLU	`max(alpha*x, x)`	unbounded	Keeps a small negative slope `alpha`
Softmax	`exp(z_i) / sum_j exp(z_j)`	each 0 to 1	Converts class logits to probabilities summing to 1

Here `x` is a scalar input, `z_i` is the logit for class `i`, `j` indexes all classes, and `alpha` is a small positive slope such as 0.01.

For numerical stability, subtract the largest logit before exponentiating. This does not change the probabilities.

Cross-entropy

For one correct class `c`:

```
L = -ln(p_c)

L = cross-entropy loss
p_c = predicted probability of the correct class
ln = natural logarithm
```

High correct-class probability gives small loss. Softmax plus cross-entropy is standard for multi-class single-label classification. Sigmoid plus binary cross-entropy is appropriate when classes are independent or multiple labels can be true.

Overfitting and regularization

Overfitting means very good training performance but poor generalization to unseen data.

Method	Mechanism	Key exam point
Data augmentation	Create label-preserving variations	Adds diversity; transformation must preserve meaning
Dropout	Randomly zero activations during training	Reduces co-adaptation; disabled/scaled appropriately at inference
Early stopping	Stop when validation performance stops improving	Uses validation, not test data, for the decision
L1 penalty	Add absolute parameter magnitudes	Encourages exact zeros and sparsity

Method	Mechanism	Key exam point
L2 penalty	Add squared parameter magnitudes	Smoothly shrinks weights; weight decay is closely related

```
L_total_L1 = L_data + lambda * sum_i abs(theta_i)
L_total_L2 = L_data + lambda * sum_i theta_i^2

L_data = original task loss
lambda = nonnegative regularization strength
theta_i = the i-th trainable parameter
abs = absolute value
```

Too much regularization causes underfitting. The test set should remain untouched until final evaluation.

Part V - Recurrent models

Why recurrent neural networks

RNNs handle ordered data by carrying a hidden state through time. Parameters are shared at every time step, so the same transition processes a variable-length sequence.

```
h_t = tanh(W_xh x_t + W_hh h_(t-1) + b_h)
y_t = softmax(W_hy h_t + b_y)

t = time-step index
x_t = input vector at time t
h_t = hidden state at time t
h_(t-1) = previous hidden state
W_xh = input-to-hidden weight matrix
W_hh = recurrent hidden-to-hidden weight matrix
b_h = hidden bias
W_hy = hidden-to-output weight matrix
b_y = output bias
y_t = output probability vector
```

Common layouts: one-to-one, one-to-many, many-to-one, and many-to-many. A bidirectional RNN uses both past-to-future and future-to-past context, so it is unsuitable for strictly streaming situations unless future input is available.

RNN parameter count

For a simple RNN with input size `n_x`, hidden size `n_h`, output size `n_y`:

```
recurrent_cell = n_h * (n_x + n_h + 1)
output_layer = n_y * (n_h + 1)
total = recurrent_cell + output_layer
```

The count does not multiply by sequence length because the same weights are reused.

BPTT and gradient problems

Backpropagation through time unfolds the RNN across steps and applies the chain rule. Repeated multiplication by derivatives can cause:

- **Vanishing gradient:** values shrink toward zero, so early steps learn slowly and long dependencies are lost.
- **Exploding gradient:** values grow extremely large, causing unstable updates or numerical overflow.

Mitigations include LSTM/GRU gates, suitable initialization, gradient clipping for exploding gradients, normalization, residual connections, truncated BPTT, and careful learning rates. Gradient clipping does not solve vanishing gradients.

LSTM

The cell state `c_t` is a long-term memory path. Gates are sigmoid values between 0 and 1 and act like soft switches.

```
f_t = sigmoid(W_f [h_(t-1), x_t] + b_f)
i_t = sigmoid(W_i [h_(t-1), x_t] + b_i)
c_tilde_t = tanh(W_c [h_(t-1), x_t] + b_c)
c_t = f_t .* c_(t-1) + i_t .* c_tilde_t
o_t = sigmoid(W_o [h_(t-1), x_t] + b_o)
h_t = o_t .* tanh(c_t)
```

Symbols:

- `f_t`: forget gate; how much old cell state remains.
- `i_t`: input gate; how much candidate memory is written.
- `c_tilde_t`: candidate cell content.
- `c_t`: new cell state.
- `o_t`: output gate; how much cell information becomes hidden output.
- `[a,b]`: concatenate vectors `a` and `b`.
- `.*`: element-wise multiplication.

```
LSTM_parameters = 4 * n_h * (n_x + n_h + 1)
```

The factor 4 is for forget, input, candidate, and output transformations. Add a separate output layer if the question includes one.

GRU

The GRU merges memory and hidden state and has fewer gates than LSTM. Use the convention taught in the supplied slide:

```
r_t = sigmoid(W_r [x_t, h_(t-1)] + b_r)
h_tilde_t = tanh(W_h [x_t, r_t .* h_(t-1)] + b_h)
z_t = sigmoid(W_z [x_t, h_(t-1)] + b_z)
h_t = (1 - z_t) .* h_(t-1) + z_t .* h_tilde_t
```

- `r_t`: reset gate; controls how much old state helps form the candidate.
- `z_t`: update gate; in this convention, high `z_t` favors the candidate.
- `h_tilde_t`: candidate hidden state.

Some books reverse the two terms in the final update. State the convention you are using; then calculate consistently.

```
GRU_parameters = 3 * n_h * (n_x + n_h + 1)
```

Model	Strength	Limitation
Simple RNN	Small and conceptually simple	Weak long-term memory
LSTM	Explicit cell state and fine control	Four transformations, more parameters
GRU	Fewer parameters and often faster	Less explicit control than LSTM

Part VI - Attention, Transformer and Vision Transformer

Scaled dot-product attention

Attention lets each query retrieve a weighted combination of values based on similarity to keys.

```
Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V
```

- `Q`: query matrix; what each position is looking for.
- `K`: key matrix; what each position offers for matching.
- `V`: value matrix; information returned after matching.
- `K^T`: transpose of `K` so dot products can be computed.
- `d_k`: dimension of each key/query vector.
- `sqrt(d_k)`: scaling that prevents large dot products from making softmax too sharp.
- Softmax is applied across keys for each query, so each row of weights sums to 1.

Shape check for one head:

```
Q: n_q x d_k
K: n_k x d_k
V: n_k x d_v
QK^T: n_q x n_k
output: n_q x d_v
```

Multi-head attention

Each head has learned projections of Q, K, and V. Different heads can focus on different relations. Head outputs are concatenated and multiplied by output projection `W_O`.

```
head_i = Attention(Q W_Q_i, K W_K_i, V W_V_i)
MHA = Concat(head_1, ..., head_h) W_O
```

Here `h` is the number of heads and `i` indexes a head. Usually `d_model` is divisible by `h`, with per-head dimension `d_k = d_model / h`.

Transformer encoder flow

A defensible post-normalization description is:

1. Input embeddings plus positional encodings.
2. Multi-head self-attention.
3. Residual addition and layer normalization.
4. Position-wise feed-forward network.
5. Residual addition and layer normalization.

Modern implementations may use pre-normalization. If the diagram in the question shows a particular order, follow it. Residual connections preserve information and gradient flow; layer normalization stabilizes activations; the feed-forward network transforms each position independently with shared weights.

Why positional encoding

Self-attention alone is insensitive to token order. Positional information tells the model which token or image patch came first, next, above, or below. It can be fixed or learned.

Vision Transformer patch arithmetic

An image is divided into patches, each flattened and linearly projected to an embedding. A classification token may be prepended, and positional embeddings are added.

```
N = (H / P_h) * (W / P_w)
patch_vector_length = P_h * P_w * C
patch_projection_parameters = (patch_vector_length + 1) * D_e
position_parameters_with_CLS = (N + 1) * D_e
```

- `H`, `W`: image height and width.
- `P_h`, `P_w`: patch height and width.
- `C`: image channels.
- `N`: number of image patches, assuming exact divisibility and non-overlap.
- `D_e`: embedding dimension.
- `+1` inside projection count: one bias per embedding component.
- `CLS`: extra classification token, so sequence length becomes `N + 1`.

Example: `256 x 256 x 3`, `32 x 32` patches:

```
N = (256 / 32)^2 = 8^2 = 64 patches
patch vector = 32 * 32 * 3 = 3072 values
sequence length with CLS = 65 tokens
```

ViT versus CNN

CNN	ViT
Strong local and translation-aware inductive bias	Global interactions available through self-attention
Often data-efficient on smaller vision datasets	Usually benefits from larger data or pretraining
Convolution cost grows roughly with pixels and kernel	Standard attention cost grows quadratically with token count
Hierarchical local features naturally emerge	Patch size controls sequence length and fine-detail loss

Smaller patches preserve detail but produce more tokens and much more attention computation. Larger patches are cheaper but may miss small patterns.

Part VII - Autoencoders and generative adversarial networks

Autoencoder

An encoder maps input `x` to latent code `z`; a decoder reconstructs `x_hat`.

```
z = Encoder(x)
x_hat = Decoder(z)
MSE = (1/n) * sum_i (x_i - x_hat_i)^2
```

- `z`: compressed representation.

- `x_hat`: reconstruction.
- `n`: number of reconstructed values.

Uses include dimensionality reduction, denoising, anomaly detection, compression, and representation pretraining. A bottleneck or regularization prevents a trivial copy.

A variational autoencoder predicts distribution parameters such as mean and variance, samples a latent code, and combines reconstruction loss with KL-divergence regularization. This produces a smoother, sampleable latent space.

GAN components and objective

- Generator `G(z)` turns random noise `z` into a fake sample.
- Discriminator `D(x)` outputs how real a sample appears, commonly between 0 and 1.
- `p_data`: real-data distribution.
- `p_z`: noise distribution.

```
min_G max_D V(D,G)
= E_x[ln D(x)] + E_z[ln(1 - D(G(z)))]
```

The discriminator maximizes correct real/fake discrimination. The generator tries to fool it. In practice, the non-saturating generator loss is often used:

```
L_G = -E_z[ln D(G(z))]
```

Alternating training loop

1. Sample real data and noise.
2. Generate fake samples.
3. Update discriminator using real labels 1 and fake labels 0.
4. Sample fresh noise.
5. Update generator through the discriminator using the goal that fake is classified real.
6. Repeat; at generation time the discriminator can be discarded.

Common failures: mode collapse, unstable oscillation, vanishing gradients, and imbalance between G and D. Remedies include careful objectives, normalization, architecture choices, data augmentation, regularization, and balanced update schedules.

Pix2Pix and CycleGAN

Feature	Pix2Pix	CycleGAN
Data	Paired input-target images	Unpaired sets from two domains
Generator	Usually U-Net	Usually residual generator
Discriminator	Conditional PatchGAN	Domain discriminators
Key losses	Adversarial plus L1 reconstruction	Adversarial plus cycle consistency
Best use	Aligned map-photo or sketch-photo pairs	Summer-winter, horse-zebra without aligned pairs

Pix2Pix's L1 term keeps output close to the paired target and tends to be less blurry than pure L2. PatchGAN judges local patches, encouraging realistic texture.

Cycle consistency requires translating there and back to recover the input:

```
L_cyc = E_x[||F(G(x)) - x||_1] + E_y[||G(F(y)) - y||_1]
```

- `G`: mapping from domain X to Y.
- `F`: mapping from Y to X.
- `||.||_1`: sum of absolute differences.

Cycle consistency encourages content preservation but does not guarantee a semantically correct one-to-one mapping.

Old-paper Pix2Pix parameter interpretation

If a simplified question says **seven convolution layers starting at 32 filters and doubling**, a transparent assumption is encoder channels `3 -> 32 -> 64 -> 128 -> 256 -> 512 -> 1024 -> 2048`, all `3 x 3`, with one bias per output channel. The total is:

```
sum_l (3 * 3 * C_in_l + 1) * C_out_l = 25164608 parameters
```

A real full U-Net also has decoder/up-convolution layers, so it would have more parameters. State that your number covers the seven stated convolutions only.

If `256 x 256` is divided into **non-overlapping** `64 x 64` patches:

```
(256 / 64)^2 = 16 patches
```

A real sliding PatchGAN output count depends on its stride, padding, and receptive field. If those are absent, explicitly use the non-overlapping interpretation.

Part VIII - Adversarial attacks and defense

Threat categories

- **Poisoning attack:** corrupt training data or labels before/during learning.
- **Evasion attack:** modify input at inference time to cause a wrong prediction.
- **White-box:** attacker knows model, parameters, or gradients.
- **Black-box:** attacker uses outputs or transferred examples without full internals.
- **Targeted:** force a chosen wrong class.
- **Untargeted:** cause any wrong class.

Attacks from the slides include adversarial noise, semantic perturbation, FGSM, BIM, and DeepFool.

FGSM

```
x_adv = clip(x + epsilon * sign(gradient_x L(theta, x, y)), valid_range)
```

- `x`: clean input.
- `x_adv`: adversarial input.
- `epsilon`: maximum perturbation size under an L-infinity interpretation.
- `L`: model loss for parameters `theta`, input `x`, true label `y`.
- `gradient_x L`: direction in input space that increases loss fastest locally.
- `sign`: replace each component by -1, 0, or +1.
- `clip`: keep pixels in their valid range such as 0 to 1.

FGSM is a one-step attack. BIM repeats smaller steps `alpha` and projects/clips back into the allowed `epsilon` region. Adversarial training includes adversarial examples during training and is one of the strongest standard defenses, though robustness is often attack-specific and costs computation.

Other defenses: preprocessing, input detection, ensembles, robust objectives, and certified methods. Gradient masking can create a false sense of security, so evaluate with adaptive attacks.

Part IX - Explainability, transfer learning and LLMs

Explainable AI

XAI makes a model's behavior understandable enough for debugging, trust, compliance, or scientific insight.

Axis	Options	Meaning
Scope	Local vs global	Explain one prediction vs overall behavior
Dependency	Model-specific vs model-agnostic	Uses internals vs treats model as a black box
Timing	Intrinsic vs post-hoc	Interpretable by design vs explanation after training

Methods in the material:

- **LIME:** fit a simple local surrogate around one input; intuitive but can be unstable.
- **SHAP:** feature contributions based on Shapley-value ideas; principled but may be expensive.

- **Permutation importance:** shuffle a feature and measure performance drop; global and model-agnostic, but correlated features complicate interpretation.
- **Partial dependence plot:** average prediction as one or two features vary; may evaluate unrealistic combinations when features correlate.
- **TreeExplainer:** efficient SHAP-style explanations for tree models.
- **CNN visualization/Grad-CAM:** heatmap of spatial regions supporting a class; coarse localization is not a causal proof.
- **Counterfactual:** smallest meaningful change that would alter the prediction.

Never say an explanation proves causality. It explains model behavior under assumptions and should be validated.

Transfer learning and domain adaptation

- **Transfer learning:** reuse knowledge from a source task/model for a target task, often by feature extraction or fine-tuning.
- **Feature extraction:** freeze most pretrained layers and train a new head; faster and safer with little target data.
- **Fine-tuning:** unfreeze some/all layers with a small learning rate; more adaptable but can overfit or forget useful features.
- **Domain adaptation:** source and target tasks are related or the same, but their input distributions differ, such as synthetic versus real images.

A strong domain-adaptation answer names the source domain, target domain, distribution shift, shared representation or alignment method, and evaluation on labeled target data. Avoid negative transfer by checking that source knowledge is relevant.

Large language models

LLMs are Transformer-based models trained on large text corpora to model token sequences. Suitable applications include:

Application	Why suitable	Main caution
Summarization	Models long-range language patterns	Can omit or invent facts
Question answering/chat	Generates context-conditioned responses	Needs grounding and safety controls
Code assistance	Learns code and natural-language relations	Generated code must be tested and reviewed
Information extraction	Flexible across formats	Validate structured output
Translation/content drafting	Strong sequence generation	Cultural nuance, bias, copyright/privacy

Terms worth recalling: tokenization, embeddings, self-attention, pretraining, fine-tuning, prompting, context window, hallucination, retrieval augmentation, and human evaluation.

Part X - Lab-linked knowledge

Lab 1: MNIST decision tree

Core steps: load images/labels, flatten each image into a feature vector, normalize if required, split train/test, fit decision tree, predict, and evaluate. A decision tree does not consume a 2D grid directly; flattening loses explicit spatial structure. Important risks are overfitting with excessive depth and leakage from test data.

Lab 2: CIFAR-10 CNN

Core steps: load RGB data, scale pixel values, one-hot encode labels if using categorical cross-entropy, build convolution/pooling blocks, flatten or global-pool, use dense softmax output, compile with an optimizer such as Adam, train with validation, dropout/augmentation/early stopping, and evaluate on the held-out test set.

Know why each step exists. For example, normalization stabilizes optimization, softmax creates class probabilities, cross-entropy penalizes low probability on the true class, and early stopping limits overfitting.

Lab 3: adversarial MNIST

Core steps: train a classifier, compute gradient of loss with respect to input, create FGSM example, clip pixels, compare clean/adversarial accuracy, then adversarially train or fine-tune and re-evaluate both. Always report the attack strength `epsilon`; robustness without an attack setting is meaningless.

Part XI - Answer-writing templates

Three-mark definition/comparison

Use three compact parts:

1. One-sentence definition.
2. Mechanism or two differences.
3. Example, benefit, or limitation.

Example for dropout: define random activation masking during training; explain reduced co-adaptation and inference behavior; finish with its role against overfitting and the risk of underfitting if too strong.

Five-mark architecture question

Use **D-F-P-A-L**:

- **D:** definition and purpose.
- **F:** flow through major blocks.
- **P:** why each special component helps.
- **A:** application or comparison.
- **L:** limitation.

Add a labeled block diagram whenever possible. A neat diagram earns clarity marks even if prose is short.

Ten-mark numerical/case question

1. List assumptions.
2. Draw or tabulate shapes/channels layer by layer.
3. Write the general formula and define symbols.
4. Substitute numbers visibly.
5. Give the result with units.
6. Interpret the result.
7. Answer the conceptual comparison separately.

High-yield one-line distinctions

Pair	Distinction
Parameter vs hyperparameter	Parameter is learned; hyperparameter is chosen before/during training
Training vs validation vs test	Fit weights; tune/stop; final unbiased estimate
Padding vs stride	Padding controls borders/size; stride controls movement/downsampling

Pair	Distinction
Kernel count vs kernel size	Count determines output channels; size determines local spatial extent
L1 vs L2	L1 promotes sparsity; L2 smoothly shrinks weights
ResNet skip vs U-Net skip	Addition within residual block vs encoder-decoder feature concatenation
RNN hidden state vs LSTM cell state	Short exposed state vs dedicated long-term memory path
Attention key vs value	Key is matched; value is retrieved
Autoencoder vs GAN	Reconstruction bottleneck vs adversarial sample generation
Pix2Pix vs CycleGAN	Paired conditional mapping vs unpaired cycle-consistent mapping
Poisoning vs evasion	Attack training pipeline vs inference input
Transfer learning vs domain adaptation	Reuse knowledge broadly vs handle source-target distribution shift
Local vs global XAI	Explain one result vs whole-model behavior

Final 15-minute recall list

- Can I calculate batches and total updates?
- Can I write the complete convolution output formula with dilation?
- Can I count convolution, dense, RNN, LSTM, and GRU parameters?
- Can I explain why pooling has zero trainable parameters?
- Can I distinguish Inception, ResNet, DenseNet, and U-Net skips?
- Can I write all LSTM and GRU update equations and name every gate?
- Can I write `softmax(QK^T/sqrt(d_k))V` and state all shapes?
- Can I calculate ViT patch count, vector length, tokens, and projection parameters?
- Can I explain autoencoder, GAN loop, Pix2Pix, and CycleGAN losses?
- Can I write FGSM and explain `epsilon`, `sign`, gradient, and clipping?
- Can I distinguish L1/L2, semantic/instance segmentation, and transfer/domain adaptation?
- Can I give one benefit and one limitation for every method?

> **If the answer to one item is no, do exactly two workbook problems of that type. Do not reread the full chapter.**

Source coverage

This guide consolidates the supplied lectures on introduction, dataset types and bias, image data and convolution, CNN models and activation functions, RNN, LSTM/GRU, attention, regularization, autoencoders/GAN, adversarial attacks, Vision Transformer/U-Net, XAI/transfer learning/LLM, all three lab assignments, and the two images of the previous examination paper.